LinkedIn Launchpad — Architecture Reference

> Generated diagrams covering system overview, user flow, backend, frontend, pipelines, and tech stack. PNG images are pre-rendered from Mermaid source files in diagrams/.

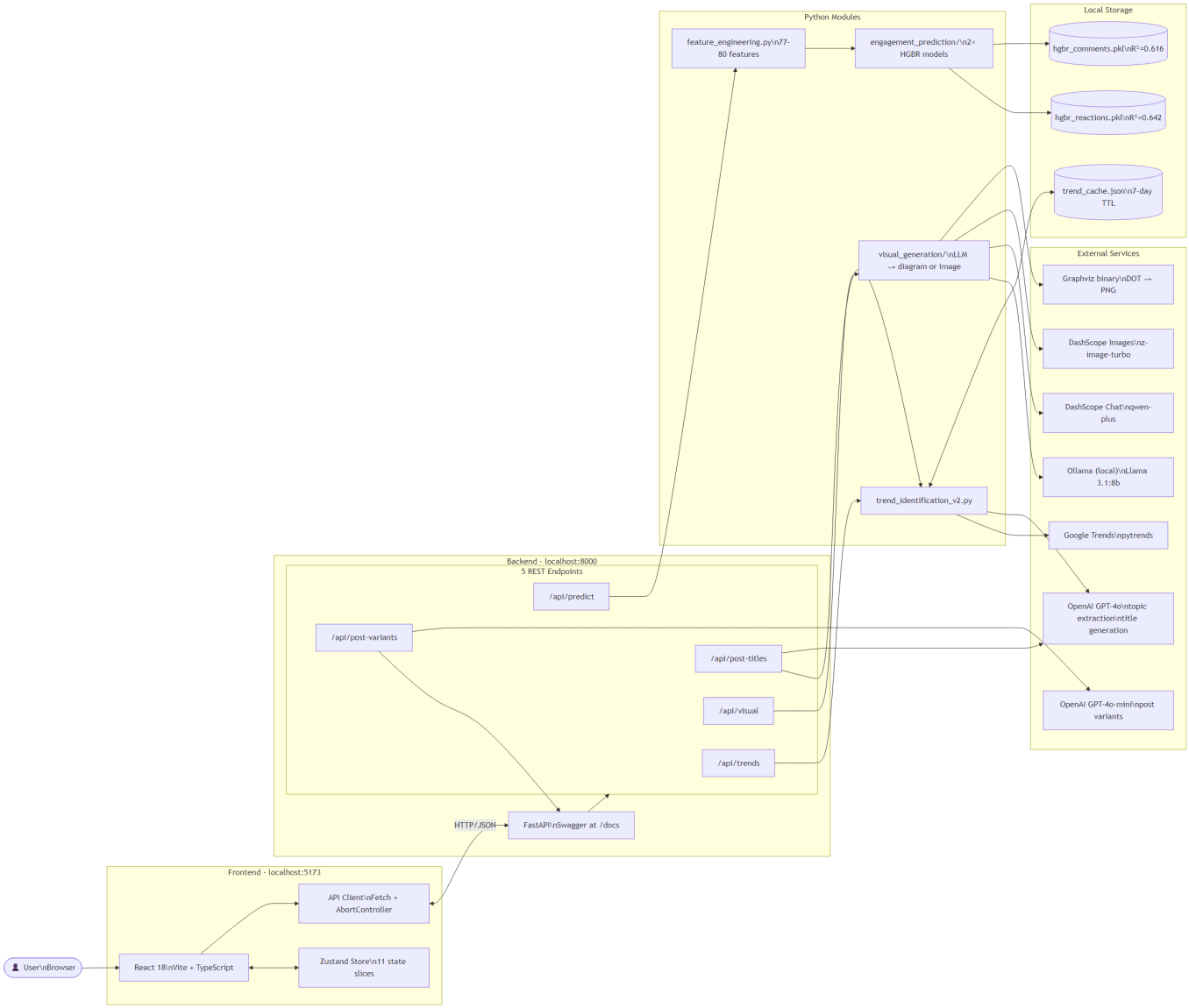
Table of Contents

1. System Overview
2. User Flow — 5-Step Wizard
3. Backend Architecture
4. Frontend Architecture
5. Visual Generation Pipeline
6. Engagement Prediction Pipeline
7. Feature Engineering
8. State Management
9. Technology Stack
10. End-to-End Data Flow
11. API Contracts
12. File Structure

---

1. System Overview

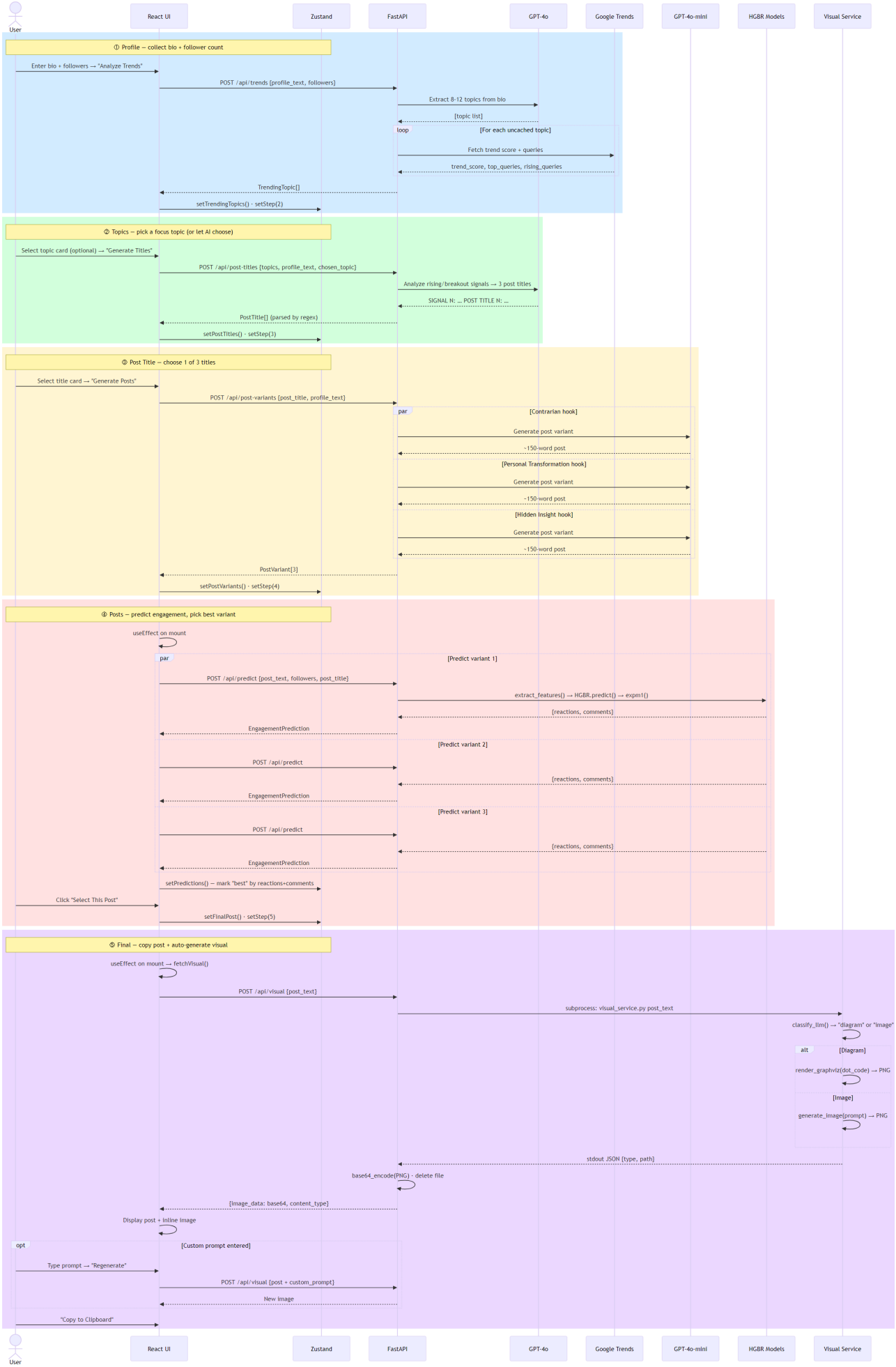
High-level view of all layers: browser → frontend → backend → modules → external services.



---

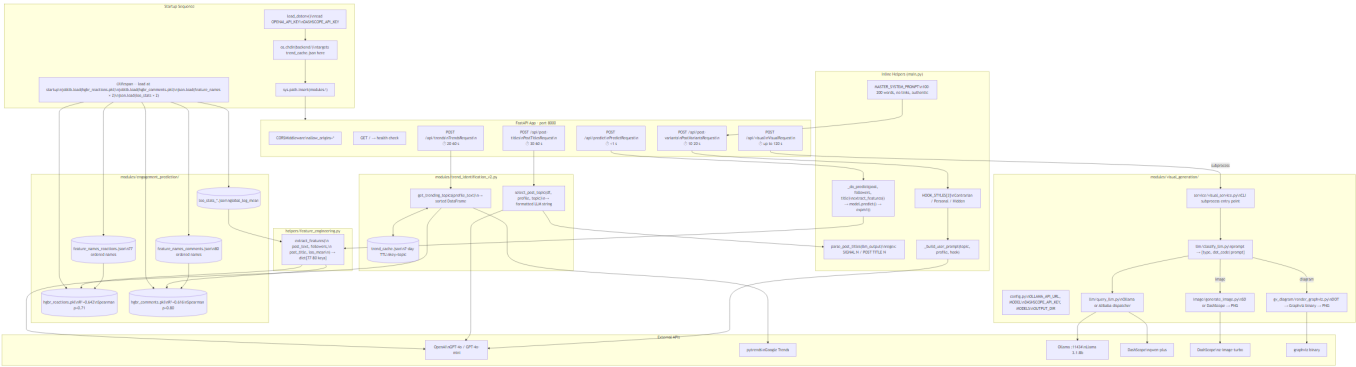
2. User Flow — 5-Step Wizard

Complete sequence from first input to final post + visual.


User
React UI
Zustand
FastAPI
GPT-4o
Google Trends
GPT-4o-mini
HGBR Models
Visual Service
① Profile — collect bio + follower count
Enter bio + followers → "Analyze Trends"
POST /api/trends {profile_text, followers}
Extract 8-12 topics from bio
[topic list]
loop
[For each uncached topic]
Fetch trend score + queries
trend_score, top_queries, rising_queries
TrendingTopic[]
setTrendingTopics() · setStep(2)
② Topics — pick a focus topic (or let AI choose)
Select topic card (optional) → "Generate Titles"
POST /api/post-titles {topics, profile_text, chosen_topic}
Analyze rising/breakout signals → 3 post titles
SIGNAL N: ... POST TITLE N: ...
PostTitle[] (parsed by regex)
setPostTitles() · setStep(3)
③ Post Title — choose 1 of 3 titles
Select title card → "Generate Posts"
POST /api/post-variants {post_title, profile_text}
par
[Contrarian hook]
Generate post variant
~150-word post
[Personal Transformation hook]
Generate post variant
~150-word post
[Hidden Insight hook]
Generate post variant
~150-word post
PostVariant[3]
setPostVariants() · setStep(4)
④ Posts — predict engagement, pick best variant
useEffect on mount
par
[Predict variant 1]
POST /api/predict {post_text, followers, post_title}
extract_features() → HGBR.predict() → expm1()
{reactions, comments}
EngagementPrediction
[Predict variant 2]
POST /api/predict
{reactions, comments}
EngagementPrediction
[Predict variant 3]
POST /api/predict
{reactions, comments}
EngagementPrediction
setPredictions() — mark "best" by reactions+comments
Click "Select This Post"
setFinalPost() · setStep(5)
⑤ Final — copy post + auto-generate visual
useEffect on mount → fetchVisual()
POST /api/visual {post_text}
subprocess: visual_service.py post_text
classify_llm() → "diagram" or "image"
alt
[Diagram]
render_graphviz(dot_code) → PNG
[Image]
generate_image(prompt) → PNG
stdout JSON {type, path}
base64_encode(PNG) · delete file
{image_data: base64, content_type}
Display post + inline image
opt
[Custom prompt entered]
Type prompt → "Regenerate"
POST /api/visual {post + custom_prompt}
New image
"Copy to Clipboard"

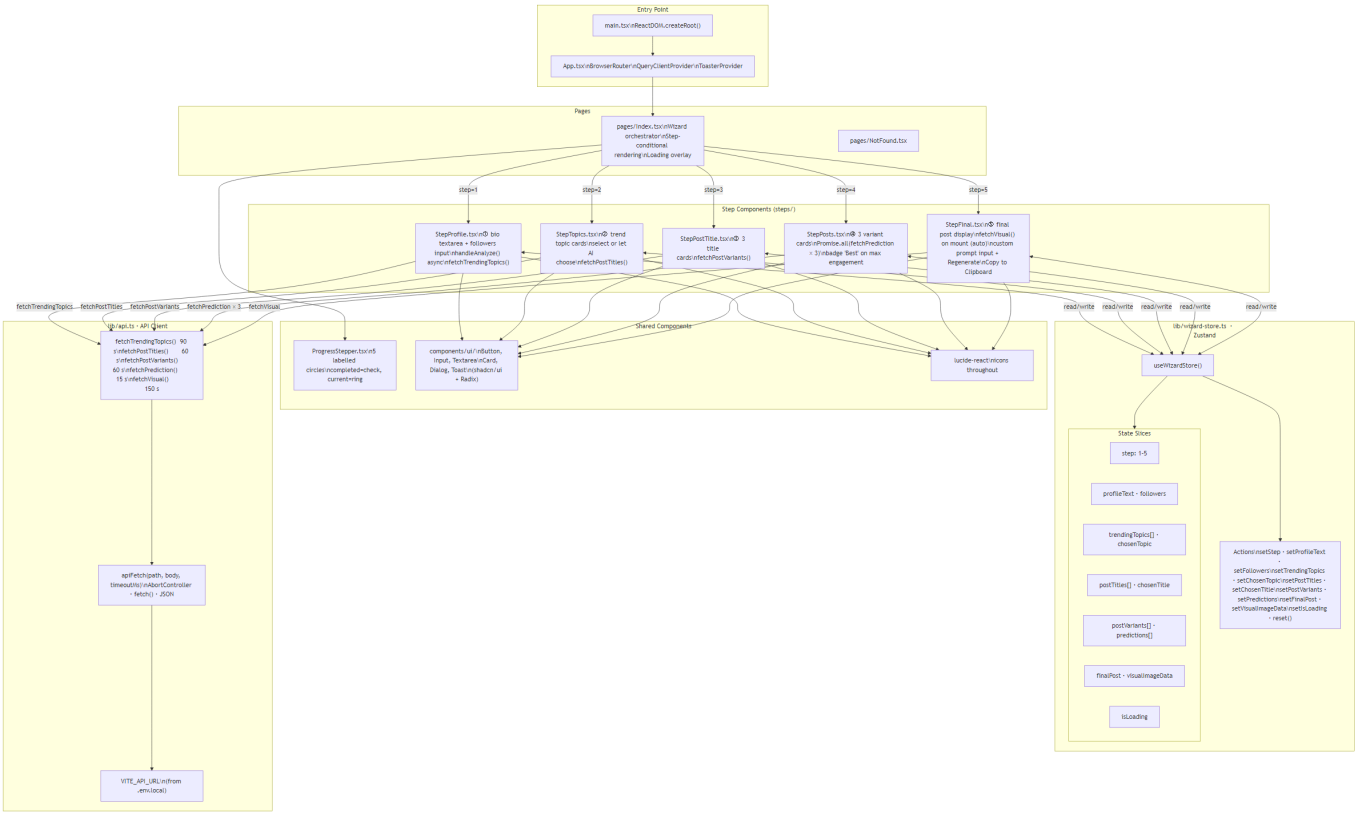
3. Backend Architecture

Internal structure of the FastAPI backend — startup, routes, helpers, and module wiring.



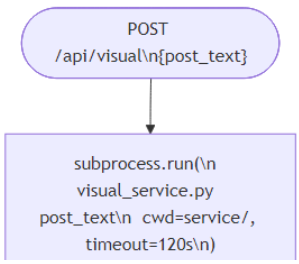
4. Frontend Architecture

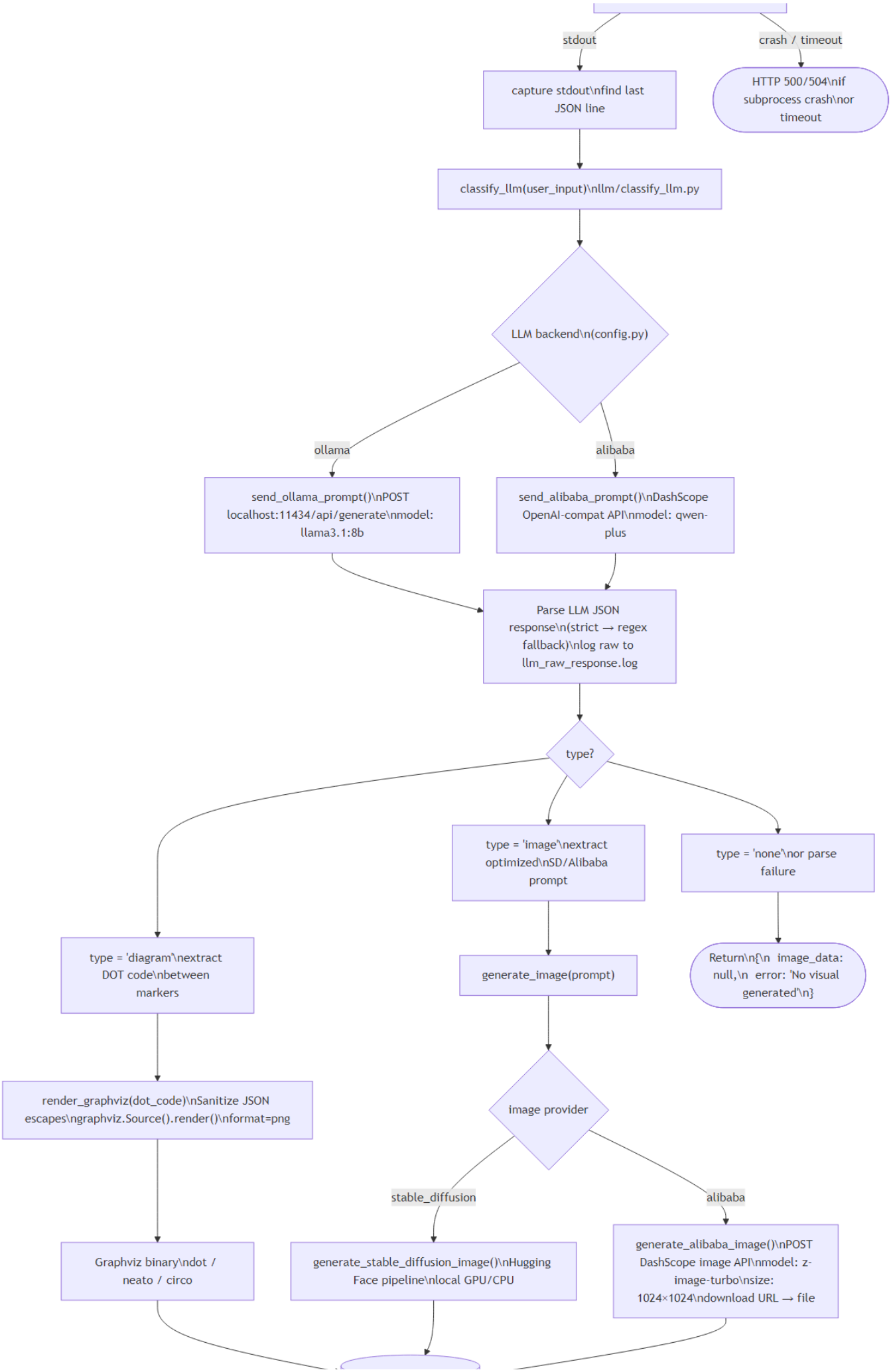
React component tree, Zustand store structure, and API client wiring.

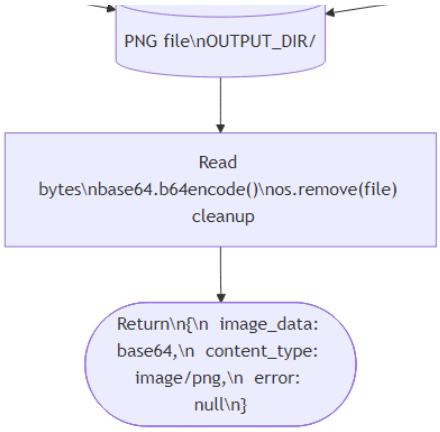


5. Visual Generation Pipeline

Detailed flow inside `visual_generation/` — from post text to base64-encoded image.



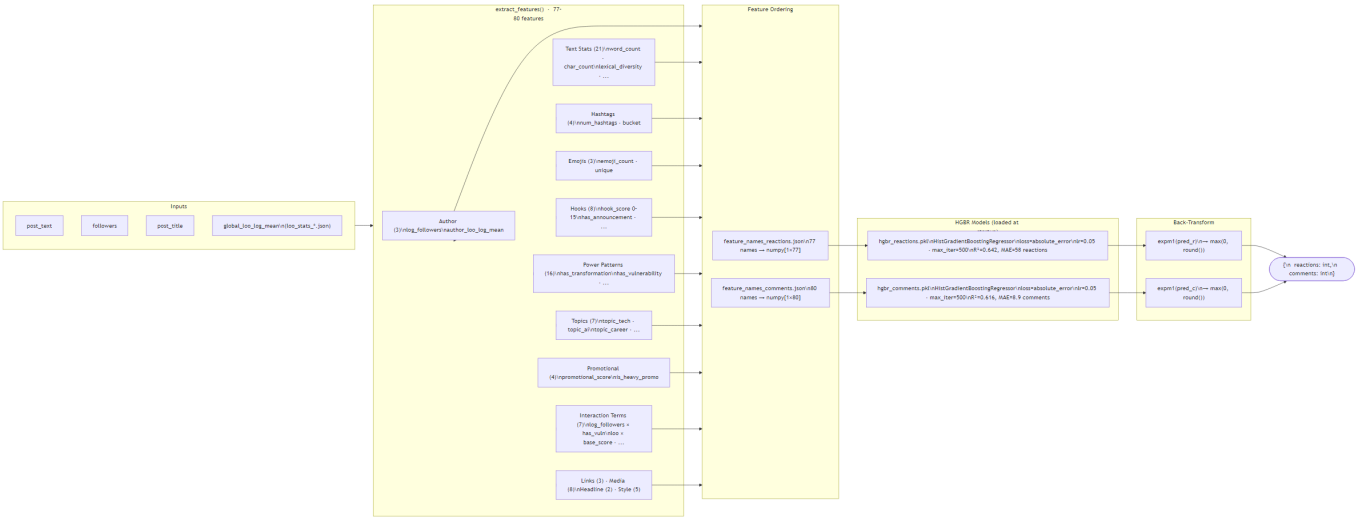

stdout
crash / timeout
capture stdout\nfind last JSON line
HTTP 500/504\nif subprocess crash\nor timeout
classify_llm(user_input)\nllm/classify_llm.py
LLM backend\n(config.py)
ollama
alibaba
send_ollama_prompt()\nPOST localhost:11434/api/generate\nmodel: llama3.1:8b
send_alibaba_prompt()\nDashScope OpenAI-compat API\nmodel: qwen-plus
Parse LLM JSON response\n(strict → regex fallback)\nlog raw to llm_raw_response.log
type?
type = 'image'\nextract optimized\nSD/Alibaba prompt
type = 'none'\nor parse failure
type = 'diagram'\nextract DOT code\nbetween markers
generate_image(prompt)
Return\n{\n image_data: null,\n error: 'No visual generated'\n}
render_graphviz(dot_code)\nSanitize JSON escapes\ngraphviz.Source().render()\nformat=png
image provider
stable_diffusion
alibaba
Graphviz binary\ndot / neato / circo
generate_stable_diffusion_image()\nHugging Face pipeline\nlocal GPU/CPU
generate_alibaba_image()\nPOST DashScope image API\nmodel: z-image-turbo\nsize: 1024×1024\ndownload URL → file



---

6. Engagement Prediction Pipeline

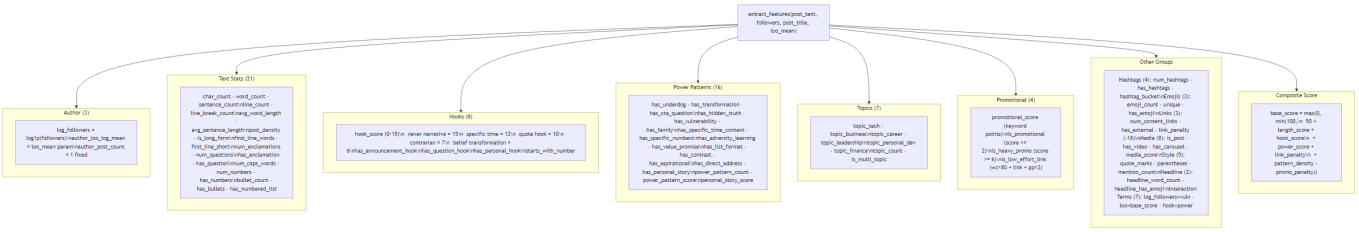
From raw post text to predicted reactions and comments.



---

7. Feature Engineering

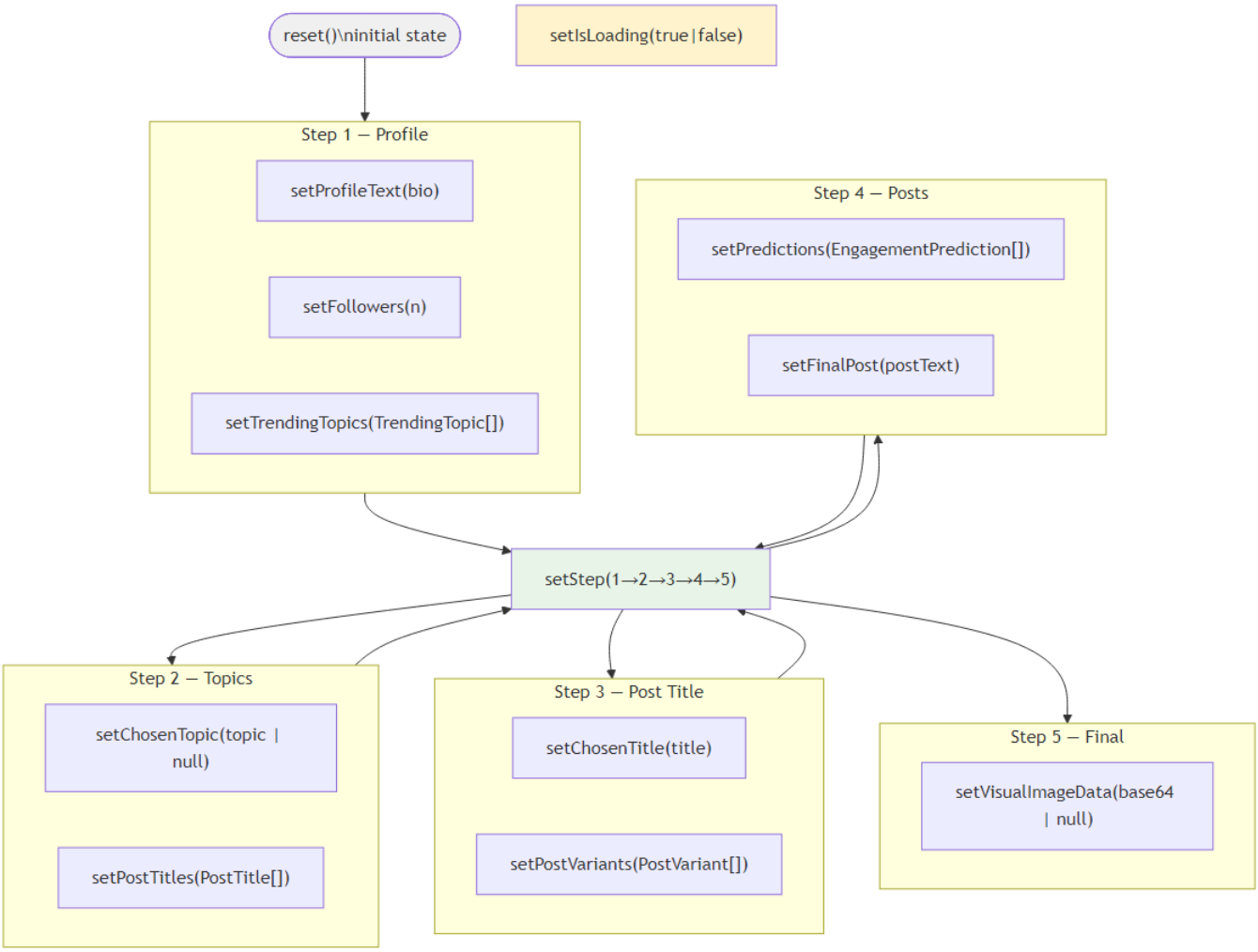
The 77–80 features extracted by `helpers/feature_engineering.py`.



---

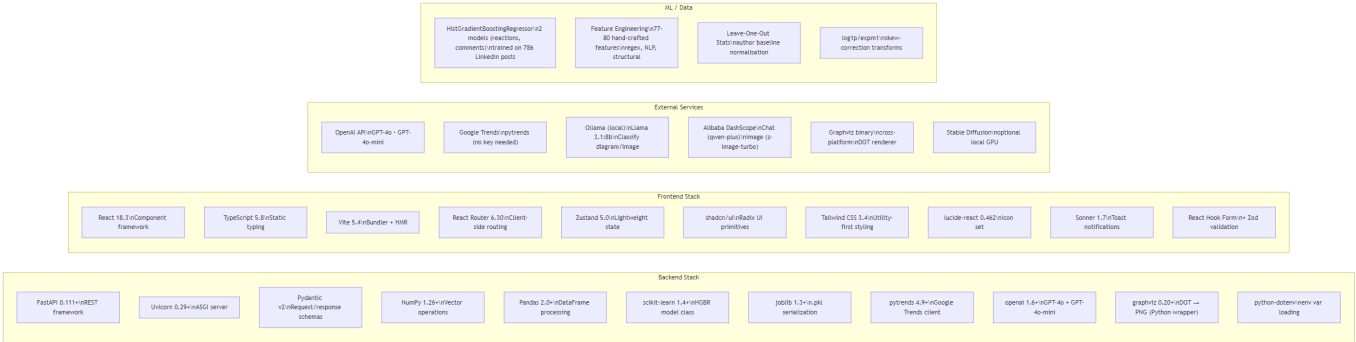
8. State Management

How data flows through the Zustand store across all 5 steps.



---

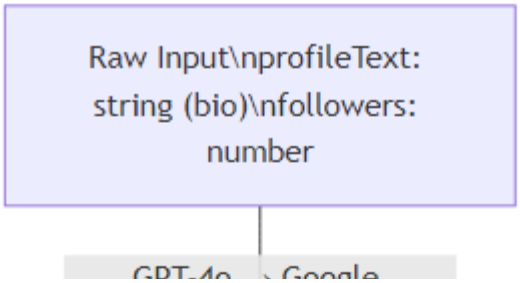
9. Technology Stack

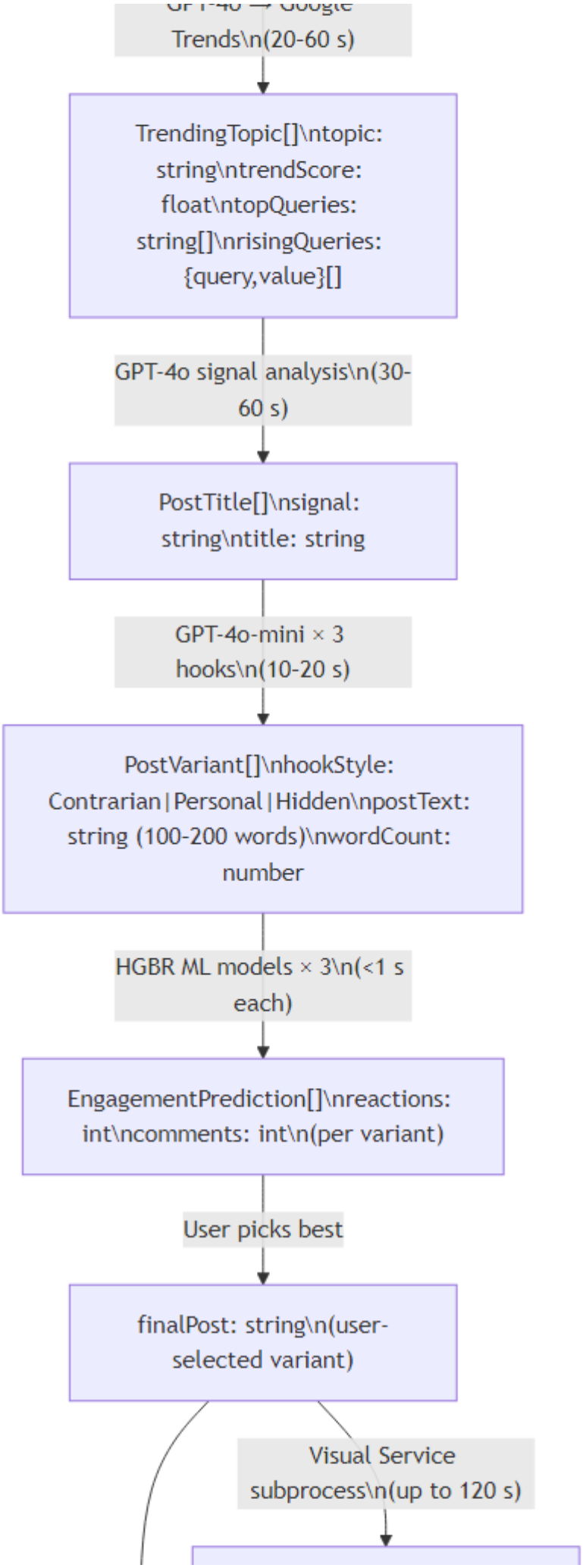


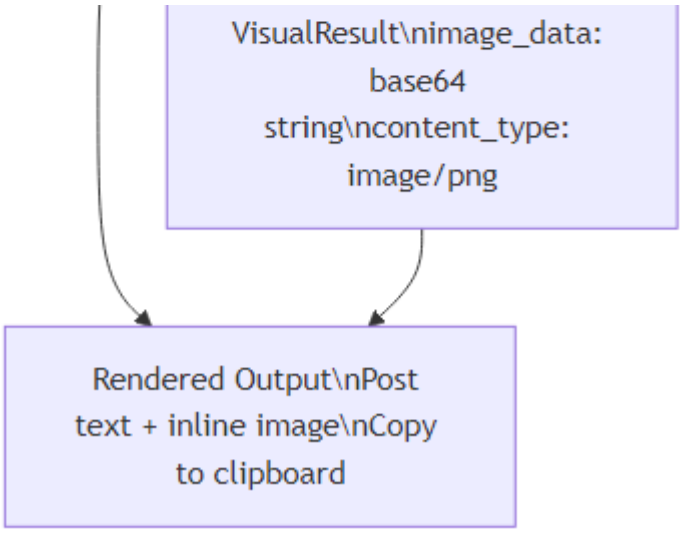
---

10. End-to-End Data Flow

Data transformations at each stage, from raw user text to final LinkedIn post.




GPT-4o → Google Trends\n(20-60 s)
TrendingTopic[]\ntopic: string\ntrendScore: float\ntopQueries: string[]\nrisingQueries: {query,value}[]
GPT-4o signal analysis\n(30-60 s)
PostTitle[]\nsignal: string\ntitle: string
GPT-4o-mini × 3 hooks\n(10-20 s)
PostVariant[]\nhookStyle: Contrarian|Personal|Hidden\npostText: string (100-200 words)\nwordCount: number
HGBR ML models × 3\n(<1 s each)
EngagementPrediction[]\nreactions: int\ncomments: int\n(per variant)
User picks best
finalPost: string\n(user-selected variant)
Visual Service subprocess\n(up to 120 s)



---

11. API Contracts

All Endpoints

Method	Path	Request Body	Response	Timeout
GET	/	—	{status, service}	—
POST	/api/trends	{profile_text, followers}	TrendingTopic[]	90 s
POST	/api/post-titles	{trending_topics[], profile_text, chosen_topic?}	PostTitle[]	60 s
POST	/api/post-variants	{post_title, profile_text}	PostVariant[]	60 s
POST	/api/predict	{post_text, followers, post_title}	{reactions, comments}	15 s
POST	/api/visual	{post_text}	VisualResult	150 s

Pydantic Schemas (Backend)

```
TrendsRequest        { profile_text: str, followers: int = 1000 }
PostTitlesRequest    { trending_topics: list[TrendingTopicIn], profile_text: str,
chosen_topic: str|None }
PostVariantsRequest  { post_title: str, profile_text: str }
PredictRequest       { post_text: str, followers: int, post_title: str = "" }
VisualRequest        { post_text: str }
```

TypeScript Interfaces (Frontend)

```
TrendingTopic        { topic, trendScore, topQueries[],
risingQueries[{query,value}] }
PostTitle            { signal, title }
PostVariant          { hookStyle, postText, wordCount }
EngagementPrediction { reactions, comments }
VisualResult         { image_data: string|null, content_type: string, error:
string|null }
```

---

12. File Structure

```
Capstone/
├── ARCHITECTURE.md                          ← this file
├── RUNNING.md                               ← how to run
├── PLAN.md                                  ← implementation plan
│
├── backend/
│   ├── main.py                              5 routes + startup + helpers
│   ├── requirements.txt
│   ├── .env                                 template (committed)
│   ├── .env.secrets                         real keys (gitignored)
│   ├── .gitignore
│   ├── helpers/
│   │   ├── __init__.py
│   │   └── feature_engineering.py           77–80 feature extractor
│   └── modules/
│       ├── trend_identification_v2.py       Google Trends + GPT-4o
│       ├── engagement_prediction/
│       │   ├── hgbr_reactions.pkl           reactions model
│       │   ├── hgbr_comments.pkl            comments model
│       │   ├── feature_names_reactions.json 77 ordered feature names
│       │   ├── feature_names_comments.json  80 ordered feature names
│       │   ├── loo_stats_reactions.json     author baseline stats
│       │   ├── loo_stats_comments.json
│       │   └── metadata.json               hyperparameters + metrics
│       └── visual_generation/
│           ├── config.py                    LLM + image API settings
│           ├── service/
│           │   └── visual_service.py        subprocess CLI entry point
│           ├── llm/
│           │   ├── classify_llm.py          LLM classification logic
│           │   └── query_llm.py             Ollama / Alibaba dispatcher
│           ├── gv_diagram/
│           │   └── render_graphviz.py       DOT → PNG renderer
│           └── image/
│               └── generate_image.py        SD / Alibaba image generator
│
├── linkedin-launchpad/
│   ├── package.json
│   ├── vite.config.ts
```

```
│   ├── tailwind.config.ts
│   ├── .env.local                          VITE_API_URL=http://localhost:8000
│   └── src/
│       ├── main.tsx                        ReactDOM entry
│       ├── App.tsx                         providers + router
│       ├── pages/
│       │   ├── Index.tsx                   5-step wizard page
│       │   └── NotFound.tsx
│       ├── components/
│       │   ├── ProgressStepper.tsx         step indicator
│       │   ├── steps/
│       │   │   ├── StepProfile.tsx         step 1
│       │   │   ├── StepTopics.tsx          step 2
│       │   │   ├── StepPostTitle.tsx       step 3
│       │   │   ├── StepPosts.tsx           step 4
│       │   │   └── StepFinal.tsx           step 5
│       │   └── ui/                         shadcn/ui components
│       ├── lib/
│       │   ├── api.ts                      API client + type exports
│       │   └── wizard-store.ts             Zustand store
│       └── hooks/
│           └── use-mobile.tsx
│
└── capstone_trend_pilot/                   original Streamlit app (unchanged)
    └── streamlit/
        └── app.py                          1083-line monolith (source of truth)
```